PyTorch 101

Tensorer, bildemodeller og trening

Innhold

Del	Tema
01	Matriser og tensoroperasjoner
02	Bilder og datasett
03	Lineære modeller og prediksjoner
04	Tap, optimalisering og evaluering
05	Nevrale nettverk

Oppgaver etter hver del

Skalarer og vektorer

Skalar

$$a = 3 \in \mathbb{R}$$

Ett tall.

Vektor

$$x = \begin{bmatrix} 2 \\ 4 \\ 6 \end{bmatrix} \in \mathbb{R}^3$$

Tre komponenter i en bestemt rekkefølge.

Matriser og dimensjoner

$$X = \begin{bmatrix} 2 & 4 & 6 \\ 3 & 5 & 7 \end{bmatrix} \in \mathbb{R}^{2\times 3}$$

	Matematikk	PyTorch
Form	2×3	`[2, 3]`
Ett element	X_{ij}	`x[i, j]`
Antall elementer	$2 \cdot 3 = 6$	`x.numel()`

Rad = eksempel. Kolonne = måling.

Prikkprodukt

$$x = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}, \qquad w = \begin{bmatrix} 2 \\ 1 \\ 0 \end{bmatrix}$$

$$x^\mathsf{T} w = \sum_{j=1}^{3} x_j w_j = 1 \cdot 2 + 2 \cdot 1 + 3 \cdot 0 = 4$$

To vektorer med samme lengde → én skalar.

```
torch.dot(torch.tensor([1., 2., 3.]), torch.tensor([2., 1., 0.]))
```

Matrise–vektor-produkt

$$\underbrace{\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}}_{A\text{: } 2\times 3} \underbrace{\begin{bmatrix} 2 \\ 1 \\ 0 \end{bmatrix}}_{w\text{: } 3\times 1} = \underbrace{\begin{bmatrix} 4 \\ 13 \end{bmatrix}}_{Aw\text{: } 2\times 1}$$

Ett prikkprodukt per rad.

$$\mathbb{R}^3 \longrightarrow \mathbb{R}^2$$

Matrisemultiplikasjon

$$\underbrace{\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}}_{[2,3]} \underbrace{\begin{bmatrix} 2 & 0 \\ 1 & 1 \\ 0 & 2 \end{bmatrix}}_{[3,2]} = \underbrace{\begin{bmatrix} 4 & 8 \\ 13 & 17 \end{bmatrix}}_{[2,2]}$$

Første tall: $1 \cdot 2 + 2 \cdot 1 + 3 \cdot 0 = 4$.

$$[B, D] \mathbin{@} [D, K] \longrightarrow [B, K]$$

> Vi summerer over **D**. De to indre størrelsene må være like.

Elementvis produkt og matriseprodukt

```python
import torch

x = torch.tensor([[1., 2.], [3., 4.]])
weights = torch.tensor([[2., 0.], [1., 3.]])

x * weights   # tensor([[ 2.,  0.], [ 3., 12.]])
x @ weights   # tensor([[ 4.,  6.], [10., 12.]])
```

`*` multipliserer **element for element**.

`@` beregner **vektede summer**.

Tensorer

Antall akser	Eksempel	Form
0	Skalar	`[]`
1	Vektor	`[3]`
2	Matrise	`[2, 3]`
3	Fargebilde	`[3, H, W]`
4	Gruppe med fargebilder	`[B, 3, H, W]`

En tensor i PyTorch er en flerdimensjonal samling tall.

Hva PyTorch gir oss

Del	Oppgave
`torch.Tensor`	Tall og tensoroperasjoner på CPU eller GPU
`torch.autograd`	Automatisk gradientberegning
`torch.nn`	Lag, modeller og tapsfunksjoner
`torch.optim`	Oppdatering av modellparametere
`torch.utils.data`	Datasett og batcher

Opprette tensorer

```
import torch

values = torch.tensor([[1., 2., 3.], [4., 5., 6.]])
zeros = torch.zeros(2, 3)
ones = torch.ones(2, 3)
random_values = torch.randn(2, 3)
sequence = torch.arange(6)
```

`tensor` : fra data · `zeros` / `ones` : fra form · `randn` : tilfeldige tall

Form, datatype og enhet

```
values = torch.tensor([[1., 2., 3.], [4., 5., 6.]])
values.shape    # torch.Size([2, 3])
values.dtype    # torch.float32
values.device   # device(type='cpu')
```

Egenskap	Betydning
`shape`	Størrelse langs hver akse
`dtype`	Hvordan tallene representeres
`device`	Hvor tensoren lagres og beregnes

Indeksering og akser

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

Uttrykk	Resultat	Form
`values[0, 1]`	`2.`	`[]`
`values[0]`	`[1., 2., 3.]`	`[3]`
`values[:, 1]`	`[2., 5.]`	`[2]`
`values[0:1]`	`[[1., 2., 3.]]`	`[1, 3]`

Heltallsindeks fjerner en akse. Et utsnitt beholder den.

Reduksjoner langs en akse

$$\begin{bmatrix} 2 & 4 & 6 \\ 3 & 5 & 7 \end{bmatrix}$$

```
measurements.mean(dim=1)
# tensor([4., 5.])

measurements.mean(dim=0)
# tensor([2.5, 4.5, 6.5])
```

`dim=1`: gjennomsnitt innen hver rad.

`[2, 3] → [2]`

`dim=0`: gjennomsnitt nedover hver kolonne.

`[2, 3] → [3]`

> `dim` angir aksen vi reduserer. Med `keepdim=True` beholdes den med størrelse 1.

Broadcasting

```
row_means = measurements.mean(dim=1, keepdim=True)
# tensor([[4.], [5.]])     form [2, 1]

centered = measurements - row_means
# tensor([[-2., 0., 2.], [-2., 0., 2.]])
```

$$[2, 3] - [2, 1] \longrightarrow [2, 3]$$

Sammenlign formene **fra høyre**. Størrelsene må være like, eller én må være 1.

> Hvorfor feiler `[2, 3] - [2]`?

Transponering og reshape

```
weights = torch.tensor([
    [1., 2., 3.],
    [4., 5., 6.],
])
weights.T
# [[1., 4.],
#  [2., 5.],
#  [3., 6.]]
```

`.T` bytter rader og kolonner i en matrise.

```
weights.reshape(3, 2)
# [[1., 2.],
#  [3., 4.],
#  [5., 6.]]
```

`reshape` gir tallene en ny form i leserekkefølge.

Lik form betyr ikke lik beregning.

Samle og dele opp tensorer

```
first = torch.tensor([1., 2., 3.])
second = torch.tensor([4., 5., 6.])

torch.stack([first, second]).shape   # [2, 3]
torch.cat([first, second]).shape     # [6]
first.unsqueeze(0).shape            # [1, 3]
```

`stack`: ny akse · `cat`: eksisterende akse

`unsqueeze`: legg til akse · `squeeze`: fjern akse med størrelse 1

Oppgaver 01 · Tensorer

`oppgaver/01_tensorer.ipynb`

Tensorer · indeksering · reduksjoner · broadcasting · matriseprodukt

$$[8, 3] \mathbin{@} [3, 5] + [5] \longrightarrow ?$$

Bilder som matriser

0.0	0.0	0.0	0.0
0.0	0.4	0.8	0.0
0.0	0.8	1.0	0.0
0.0	0.0	0.0	0.0

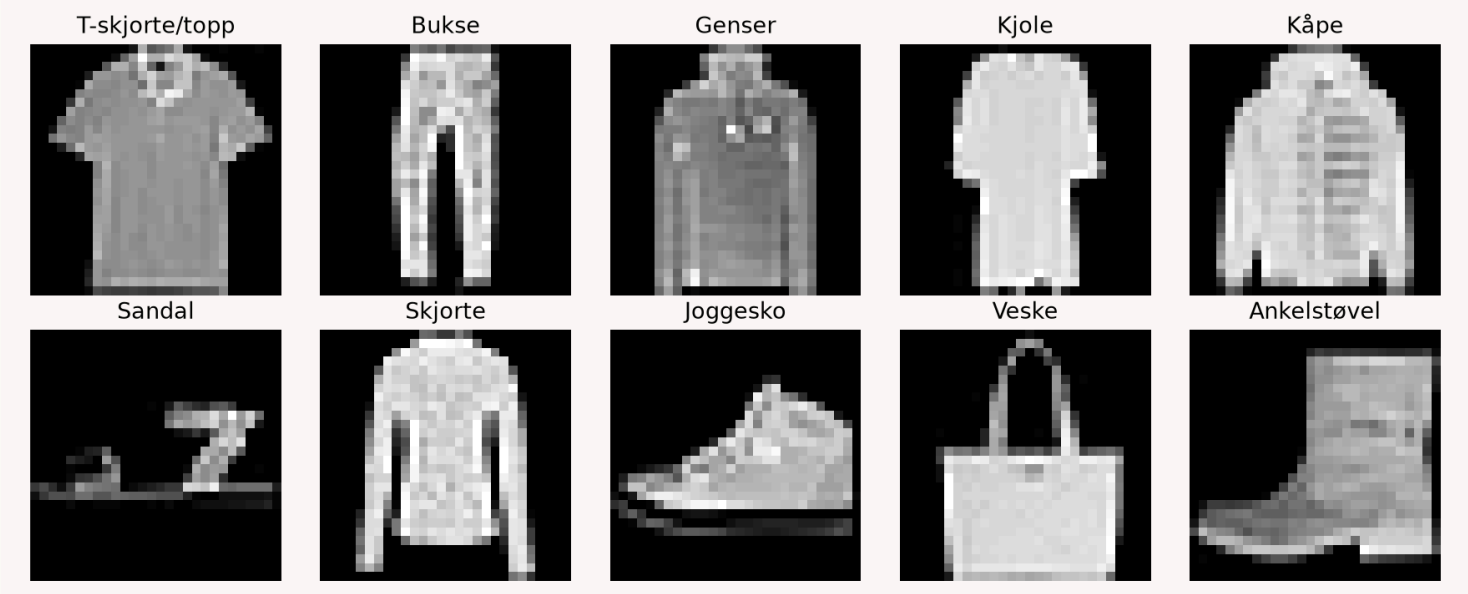
Én piksel = én lysstyrke i et gråtonebilde.

0 er svart. 1 er hvitt.

`image[row, column]` velger én piksel i denne matrisen.

Et RGB-bilde har tre kanaler: rødt, grønt og blått.

Fashion-MNIST



Kurset bruker faste utvalg: **20 000 treningsbilder** og **1 000 testbilder**. Hvert bilde er 28 × 28 piksler.

Bilder: Zalando Research · Fashion-MNIST · MIT

Kanaler og bildebatcher

Gråtonebilde	RGB-bilde	Batch
`[1, H, W]`	`[3, H, W]`	`[B, C, H, W]`

$$[64, 1, 28, 28] \quad \Rightarrow \quad 64 \cdot 1 \cdot 28 \cdot 28 = 50\,176 \text{ tall}$$

Etikettene ligger separat: `[64]`.

Dataset og transformasjoner

Indeks `i` → Les eksempel → Transformasjon → `(image, label)`

Operasjon	Betydning
`len(dataset)`	Antall eksempler
`dataset[i]`	Ett eksempel med tilhørende fasit

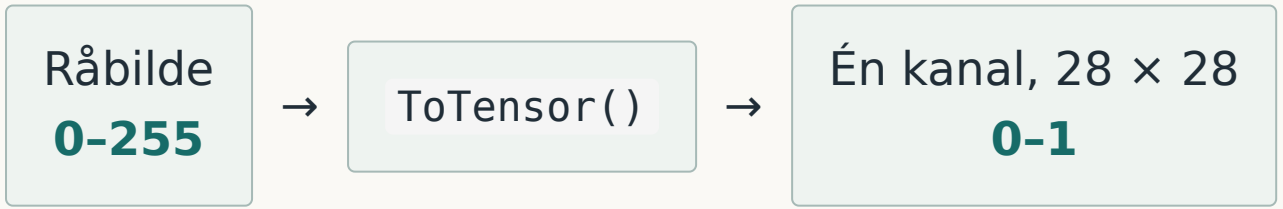
Transformasjoner kan endre datatype, skala eller bildestørrelse.

Datasett i PyTorch

```python
from pytorch_101.utils import CLASS_NAMES, download_fashion_mnist

train_data, test_data = download_fashion_mnist()
image, label = train_data[0]

image.shape          # torch.Size([1, 28, 28])
image.dtype          # torch.float32
CLASS_NAMES[label]   # Klassenavnet; label er et heltall fra 0 til 9
```



DataLoader

```python
from torch.utils.data import DataLoader

train_loader = DataLoader(train_data, batch_size=64, shuffle=True)
images, labels = next(iter(train_loader))

images.shape   # [64, 1, 28, 28]: bilder, kanaler, høyde, bredde
labels.shape   # [64]: én riktig klasse per bilde
```

Dataset: henter ett eksempel. **DataLoader:** samler flere eksempler.

`shuffle=True` stokker rekkefølgen for hver gjennomgang.

Oppgaver 02 · Bilder

`oppgaver/02_bilder.ipynb`

Pikselverdier · visualisering · klassefordeling

Lineære modeller

For én inngang: $z = wx + b$. Med flere innganger summerer vi bidragene:

$$z = w_1 x_1 + w_2 x_2 + b$$

$$x = (2, 3), \quad w = (0.5, -1), \quad b = 0.2$$

$$z = 0.5 \cdot 2 - 1 \cdot 3 + 0.2 = \mathbf{-1.8}$$

Vekter bestemmer bidragene. **Bias** flytter skåren med et konstantledd.

Klassifikasjon

$$f_\theta : \mathbb{R}^{784} \longrightarrow \mathbb{R}^{10}$$

Symbol	Betydning
x	Pikselverdier for ett bilde
θ	Modellens parametere
$z = f_\theta(x)$	Én skår per klasse
$\hat{y} = \arg\max_k z_k$	Predikert klasse

Fasiten y er en klasseindeks fra 0 til 9.

Fra bilder til klasseskårer

Bilder	→	Flate bilder	→	Klasseskårer
`[B, 1, 28, 28]`		`[B, 784]`		`[B, 10]`

```
flat_images = images.flatten(start_dim=1)
weights = torch.randn(784, 10) * 0.01
bias = torch.zeros(10)
scores = flat_images @ weights + bias
```

$$[B, 784] \text{ @ } [784, 10] + [10] \longrightarrow [B, 10]$$

Prediksjon og nøyaktighet

```
scores = torch.tensor([[0.2, 2.1, -0.4], [3.0, 0.5, 1.2]])
labels = torch.tensor([1, 2])
predictions = scores.argmax(dim=1)       # tensor([1, 0])
correct = predictions == labels          # tensor([True, False])
accuracy = correct.float().mean().item() # 0.5
```

Nøyaktighet = antall riktige / antall eksempler.

Logits og softmax

$$p_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

Klasse	Logit z_k	Softmax p_k
0	0	0.09
1	2	0.67
2	1	0.24

Logits er rå skårer. Softmax gir positive verdier som summerer til 1.

Moduler og parametere

`nn.Module` er grunnklassen for lag og modeller.

Del	Rolle
Parametere	Tensorer som læres fra data
`forward`	Beregningen fra inngang til utgang
`model(x)`	Kjør modellen på `x`
`model.parameters()`	Hent parameterne til optimizer

`nn.Sequential` er en modul som kjører andre moduler etter hverandre.

nn.Linear og nn.Sequential

```
from torch import nn

model = nn.Sequential(
    nn.Flatten(),
    nn.Linear(784, 10),
)
scores = model(images)
```

7 850 parametere: $784 \cdot 10 + 10$. Før trening er vektene tilfeldige.

Oppgaver 03 · Prediksjoner

`oppgaver/03_prediksjoner.ipynb`

Vektede summer · batcher · klasseskårer · `nn.Linear`

Trening som et optimeringsproblem

$$\theta^* = \arg\min_{\theta} \frac{1}{N} \sum_{i=1}^{N} \ell\big(f_\theta(x_i), y_i\big)$$

Størrelse	Under trening
Bilder x_i og fasit y_i	Gitt av datasettet
Parametere θ	Justeres
Tap ℓ	Måler avviket fra fasiten

I praksis tar vi mange små oppdateringssteg med batcher.

Forward og backward



Forward pass: regn fra bilder til skårer og tap.

Backward pass: beregn hvordan tapet påvirkes av hver parameter.

Oppdatering: bruk gradientene til å justere parametrene.

Tap og nøyaktighet

Fasit: klasse 1. To prediksjoner med samme vinnerklasse:

	Sannsynligheter	Riktig?	$-\log(p_y)$
A	`[0.4, 0.6]`	Ja	0.51
B	`[0.1, 0.9]`	Ja	0.11

Nøyaktigheten er lik. Tapet er lavere for B.

For ett eksempel: $\ell = -\log(p_y)$.

Kryssentropi i PyTorch

```
loss_fn = nn.CrossEntropyLoss()
loss = loss_fn(scores, labels)
```

Inndata	Form og innhold
`scores`	`[B, 10]`, rå skårer som desimaltall
`labels`	`[B]`, riktige klasseindekser som `int64`
`loss`	Ett tall, gjennomsnitt over batchen

> Send rå skårer direkte til `CrossEntropyLoss`. Den håndterer normaliseringen selv.

Gradient og læringsrate

For én parameter w gir $\frac{\partial L}{\partial w}$ den lokale endringsraten til tapet.

$$w_{\text{ny}} = w - \eta \frac{\partial L}{\partial w}$$

Enkelt eksempel på gradient descent:

$$w = 1.0, \qquad \frac{\partial L}{\partial w} = 2.0, \qquad \eta = 0.1 \quad \Rightarrow \quad w_{\text{ny}} = 0.8$$

η er læringsraten. I oppgavene bruker vi **Adam**.

Én parameteroppdatering

```
optimizer.zero_grad()           # 1. Nullstill gamle gradienter
scores = model(images)          # 2. Beregn skårer (forward)
loss = loss_fn(scores, labels)  # 3. Sammenlign med fasiten
loss.backward()                 # 4. Beregn gradienter (backward)
optimizer.step()                # 5. Oppdater vekter og bias
```

`backward()` beregner gradienter. `step()` endrer parametrene.

Batcher og epoker

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
```

Hyperparameter	Betydning
Batchstørrelse **64**	Opptil 64 bilder per oppdatering
25 epoker	25 gjennomganger av treningsutvalget
Læringsrate **0.001**	Styrer størrelsen på Adams oppdateringer

20 000 bilder gir **313 oppdateringer per epoke**

Treningsløkken

```
training_loss = []
for epoch in range(25):
    model.train()
    loss_sum = 0.0
    for images, labels in train_loader:
        optimizer.zero_grad()
        scores = model(images)
        loss = loss_fn(scores, labels)
        loss.backward()
        optimizer.step()
        loss_sum += loss.item() * len(images)
    training_loss.append(loss_sum / len(train_data))
```

Vi vekter batchtapet med antall bilder, slik at den siste, mindre batchen teller riktig.

Evaluering

```
model.eval()
num_correct = 0
with torch.no_grad():
    for images, labels in test_loader:
        scores = model(images)
        predictions = scores.argmax(dim=1)
        num_correct += (predictions == labels).sum().item()
accuracy = num_correct / len(test_data)
```

`eval()` setter evalueringsmodus. `no_grad()` slår av gradientregistrering.

Oppgaver 04 · Trening

`oppgaver/04_lineaer_klassifikator.ipynb`

Tren den lineære modellen. Tegn tapskurven. Mål testnøyaktigheten.

Nevrale nettverk

Piksler **784** → Linear + ReLU **128** → Linear **10 skårer**

$$h = \mathrm{ReLU}(xW_1 + b_1), \qquad z = hW_2 + b_2$$

Hvert skjult nevron beregner en vektet sum, så en aktivering.

Aktiveringsfunksjoner

$$\mathrm{ReLU}(a) = \max(0, a)$$

```
values = torch.tensor([-2., -0.5, 0., 1., 3.])
torch.relu(values)  # tensor([0., 0., 0., 1., 3.])
```

Uten aktivering kan to lineære lag slås sammen:

$$(xW_1 + b_1)W_2 + b_2 = x(W_1W_2) + (b_1W_2 + b_2)$$

> Ulineære aktiveringsfunksjoner gjør det mulig å lære mer enn én samlet lineær transformasjon.

Nettverket i PyTorch

```
model = nn.Sequential(
    nn.Flatten(),          # [B, 1, 28, 28] → [B, 784]
    nn.Linear(784, 128),   # [B, 784] → [B, 128]
    nn.ReLU(),            # [B, 128] → [B, 128]
    nn.Linear(128, 10),    # [B, 128] → [B, 10]
)
```

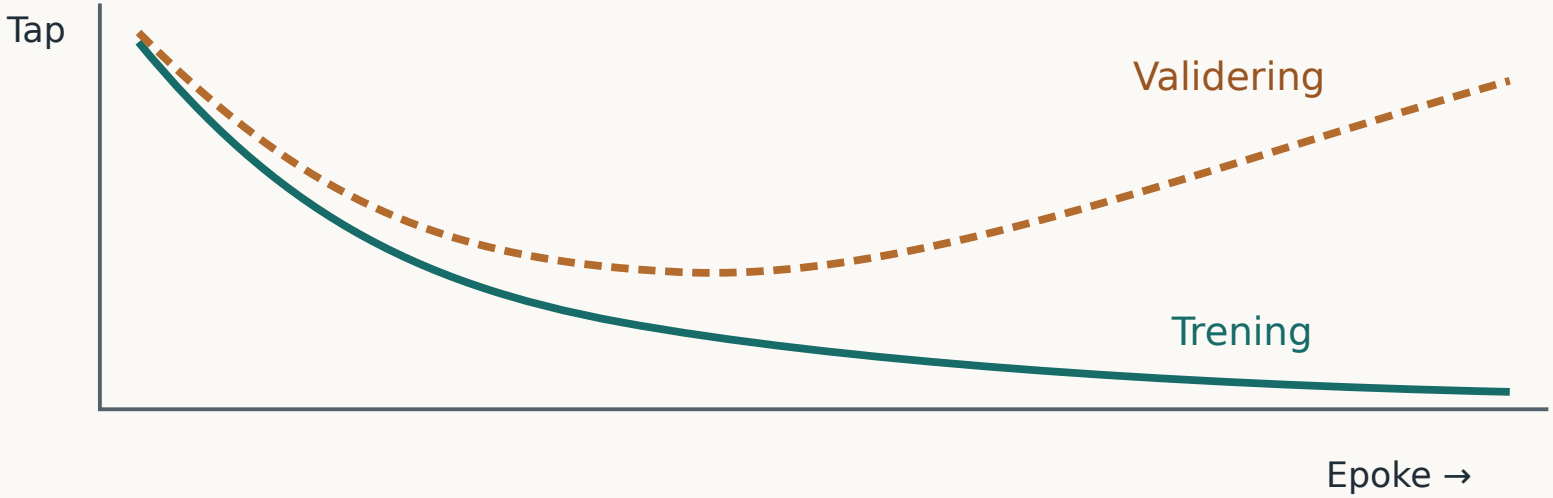
101 770 parametere: $(784 \cdot 128 + 128) + (128 \cdot 10 + 10)$

Trening, validering og test

Data	Brukes til
Trening	Lære vekter og bias
Validering	Velge innstillinger og modell
Test	Sluttevaluering

I kurset: faste hovedinnstillinger og separate trenings- og testutvalg.

Overfitting



Treningstap alene er ikke nok til å si hvor god modellen er på usett data.

class: exercise

Oppgaver 05 · Nevrale nettverk

`oppgaver/05_nevralt_nettverk.ipynb`

Trening · sammenligning · feilbilder · forvekslingsmatrise

Oppsummering

Begrep	Rolle
Tensor	Data og parametere
Modell	Inngang → skårer
Tapsfunksjon	Skårer og fasit → tap
`backward()`	Tap → gradienter
Optimizer	Gradienter → parameteroppdatering
Evaluering	Måle ytelse på egne datautvalg